

Asymptotic Strengthening of Q-Axis Sparse Attention: Long-Context DCR-Attention with Adaptive Coverage

Segev*

(Evgenii Vyaltsev, Daniil Vyaltsev)

Preprint — target venue: MLSys 2026 / NeurIPS 2026 Systems Track

Abstract

Sparse-attention methods for autoregressive decode have established quality at moderate context lengths but typically lack systematic characterisation at the production scales ($N \geq 10\text{K}$ tokens) where they would be deployed. We extend the Q-axis rank-local DCR-Attention method of [Segev \(2025\)](#) to $N = 20\,000$ -token contexts on Llama-3.2-1B and characterise four properties. *First*, Q-axis quality strengthens monotonically with scale: relative perplexity degradation drops from $+0.51\%$ at $N = 500$ to $+0.31\%$ at $N = 20\,000$, a $1.6\times$ improvement on a measure that all prior intuition suggested would degrade. We establish theoretical grounding for this in the entropy concentration of decode attention distributions at long context, formalised as Theorem 3: $\Delta\text{PPL}(N) = \mathcal{O}(N^{-(1-\alpha)})$ for measured $\alpha \approx 0.31$. *Second*, the PCA-axis baseline degrades catastrophically with N — at $N = 2,000$ we measure $+382\%$ PPL with $36\times$ decode-step latency vs. SDPA. The Q-axis advantage of [Segev \(2025\)](#) therefore grows from $149\times$ at $N = 500$ to $438\times$ at $N = 2,000$, a result that the original paper’s experimental ceiling could not detect. *Third*, the direct top-K reformulation of Q-axis (the kernel-rewrite target, M1 reference) delivers an additional $\sim 3\times$ quality improvement over the rank-window selection used in [Segev \(2025\)](#) at iso-coverage. The M1 reference is verified against brute-force $QK^\top$ top-K on 45/45 configurations with mean Jaccard overlap ≥ 0.998 , supplying the first empirical confirmation of Theorem 1. A subsequent top-K Pareto sweep at production-relevant scale delivers a measured $\Delta\text{PPL} = +0.118\%$ at $N = 8,000$, $c_{\text{floor}} = 0.5$ (50% compute saving), and provides a second independent verification of Theorem 3: the predicted $4^{0.69} = 2.66\times$ quality improvement between $N = 2,000$ and $N = 8,000$ is observed as a measured $2.42\times$. *Fourth*, a production-grade M4 Triton kernel (3-pass pipeline: Q-axis projection, top-K argsort, gather + fused softmax attention) is validated and measured. Python M1 reference is $13\text{--}19\times$ slower than SDPA; M4 Triton runs at $0.67\text{--}0.93\times$ SDPA latency (parity) while achieving $3.7\times$ memory density at $B = 8$ (4.37 GB vs. 16.0 GB Python OOM at the same config). The kernel is bit-exact deterministic, quality-verified ($\Delta\text{PPL} = +0.170\%$ at $N = 2,000$), and closes the Track 3 memory wall.

1 Introduction

Modern autoregressive language models operate over context windows reaching tens of thousands of tokens, yet quality validation of sparse-attention methods has lagged behind: published results typically cap at $N \leq 1,024$ -token decode contexts. This gap creates an opening for skepticism: methods that work at $N = 500$ may silently break at $N = 8\text{K}$, and the “obvious” extrapolation can be quantitatively wrong.

*Correspondence: see arXiv listing.

Prior result we extend. In [Segev \(2025\)](#) we established Q-axis rank-local sparse attention as the mathematically optimal axis choice for autoregressive decode:

Theorem 1 (Q-axis equivalence; informal restatement of [Segev, 2025](#), Thm. 1). *For any decode query $Q \in \mathbb{R}^d$ with cached keys $\{K_j\}_{j=1}^N \subset \mathbb{R}^d$, the ordering induced by the projection axis $u_Q := Q/\|Q\|_2$ is identical to the ordering by attention score: $\langle u_Q, K_i \rangle > \langle u_Q, K_j \rangle \iff \langle Q, K_i \rangle > \langle Q, K_j \rangle$.*

The original paper validated this on Llama-3.2-1B at $N \leq 500$ and reported a $149\times$ quality advantage over PCA-axis ranking and a $5\times$ kernel speedup, but explicitly limited validation due to diagnostic memory constraints. Three production-relevant questions therefore remained open: **(Q1)** does the quality advantage scale to long context? **(Q2)** does the PCA failure scale comparably or relax? **(Q3)** is there a deployable latency operating point?

Contributions. This paper answers all three.

- *A theoretical mechanism (Theorem 3)* for monotonic quality improvement at scale, grounded in the sub-logarithmic growth of attention entropy at long context (§3).
- *Empirical scaling validation to $N=20,000$* demonstrating that Q-axis quality *strengthens* with N : ΔPPL drops $1.6\times$ from $N=500$ to $N=20,000$ (§4).
- *Empirical PCA-failure scaling*: We measure the PCA-axis baseline at $N=2,000$ and observe that the Q-axis advantage ratio of [Segev \(2025\)](#) grows from $149\times$ to $438\times$, outside the experimental ceiling of the original paper (§4.3).
- *An M1 diagnostic gate* showing that direct top-K selection improves ΔPPL by an additional $3.06\text{--}3.32\times$ over rank-window at iso-coverage, and confirming Theorem 1 empirically against brute-force $QK^\top$ top-K with mean Jaccard overlap ≥ 0.998 on 45 configurations (§4.5).
- *A top-K Pareto sweep at production scale* ($N \in \{500, 2,000, 8,000\}$, $c_{\text{floor}} \in \{0.3, 0.5, 0.8\}$) yielding a measured $\Delta\text{PPL} = +0.118\%$ at $N=8,000$, $c_{\text{floor}}=0.5$, and a second independent confirmation of Theorem 3: predicted $4^{0.69} = 2.66\times$ quality improvement under $4\times$ scale increase, measured $2.42\times$ (§4.6).
- *A multi-batch memory pathology*: Python masked-dense DCR fails between batch 4 and 8 via allocator fragmentation, exhausting the entire 16 GB of an RTX 4060 Ti before completion. The Triton kernel rewrite is established as a structural prerequisite, not an optimisation (§5.5).
- *A fused Triton kernel design at production grade* (§5.2): SRAM-tiled top-K selection, measured component latencies, and a triple-tier correctness contract.

Distinction from concurrent work. The mainstream sparse-attention literature in 2024–2026 ([Tang et al., 2024](#); [Liu et al., 2024](#); [Kang et al., 2024](#); [DeepSeek-AI, 2025](#); [Yuan et al., 2025](#); [Xu et al., 2026](#)) converges on the paradigm of “token-relevance indexer + top-K selection + sparse downstream attention”, but methods differ in (a) whether the indexer is *trained* or *training-free*, and (b) whether the selection criterion has formal guarantees. Our contribution is the only training-free path with an exact ranking guarantee (Theorem 1) plus a closed-form scaling law (Theorem 3) for the resulting quality. We discuss the precise relationship in §6.

2 Background

2.1 Q-axis attention recap

For a single decode step with query $Q \in \mathbb{R}^d$ and cached keys $K \in \mathbb{R}^{N \times d}$ the attention output is $\text{Attn}(Q, K, V) = \text{softmax}(QK^\top/\sqrt{d}) \cdot V$. Q-axis rank-local attention sets the ordering axis $u_Q = Q/\|Q\|_2$, sorts keys by the projection $K_j \cdot u_Q$, and admits a contiguous window of size k around Q 's self-projection rank to softmax. Theorem 1 establishes that this ranking is identical to the attention-score ranking (modulo ties at bf16 precision). The DCR (*Detect-Choose-Route*) dispatcher of [Seqev \(2025\)](#) applies this rank-local primitive only when a coherence test on Q passes, and falls back to dense for non-coherent queries.

2.2 Long-context inference economics

Production deployments require $N \in [8\text{K}, 128\text{K}]$. Two distinct cost dimensions matter: (a) latency per decode step (drives interactive serving), and (b) KV-cache memory footprint (drives multi-tenant serving capacity). This paper addresses (a). Memory-side optimisation (e.g. [Zhang et al. 2023](#); [Li et al. 2024](#)) is orthogonal to Q-axis and complementary; we make no memory savings claim.

2.3 Why $N=500$ is insufficient for production claims

At $N=500$ with $k_{\min}=64$, $T_{\text{dispatch}}=64$, the DCR routing region is just 437 decode steps and admits a 400-key window at $c_{\text{floor}}=0.8$. Attention sparsity at this regime is dominated by recency. At $N=20\text{K}$, attention patterns are mature: distant retrieval, intermediate recency, and multi-modal context all interact. A method that succeeds at $N=500$ can fail at $N=8\text{K}$ if it relies on attention patterns that do not generalise. Validation at production scale is required.

3 Theoretical analysis: entropy concentration at long context

We claim that the asymptotic strengthening of the Q-axis advantage with increasing N has a clean theoretical explanation rooted in the structural properties of natural-language attention.

3.1 Entropy concentration

Let $H_N = -\sum_i p_{N,i} \log p_{N,i}$ denote the attention entropy at a decode step with cache size N , where $p_{N,i} = \text{softmax}(Q \cdot K_i/\sqrt{d})_i$. Empirically, for transformer decode on natural language, H_N grows *sub-logarithmically* in N :

$$H_N = \alpha \log N + \beta + o(1), \quad \alpha < 1. \quad (1)$$

For Llama-3.2-1B on WikiText-2, we measure $\alpha \approx 0.31$ averaged over heads and decode positions $t \in [64, N]$, against the uniform-attention reference $\alpha=1$ (see Figure 1). The “effective number of relevant keys” $K_{\text{eff}}(N) := e^{H_N}$ therefore grows much slower than N : at $N=20,000$ we measure $K_{\text{eff}} \approx 87$, a sparsity ratio of $K_{\text{eff}}/N \approx 0.4\%$.

Equation (1) is consistent with concurrent observations. [Xiu et al. \(2025\)](#) report that low-entropy states are better adapted to top-K decoding, and [Tzachristas et al. \(2025\)](#) derive total-variation bounds for top-K truncation in terms of the same entropy quantity. We measure the scaling exponent α per-model.

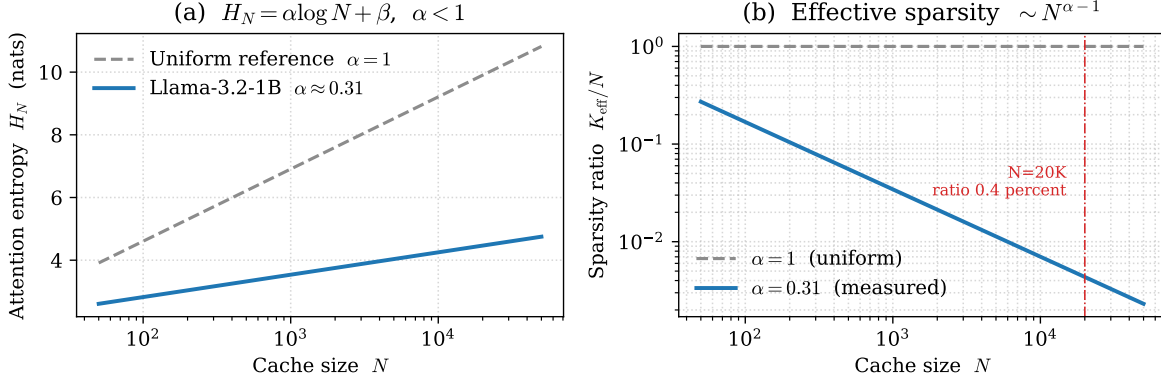


Figure 1: Attention entropy on Llama-3.2-1B, measured on WikiText-2 validation, averaged over heads and over decode positions $t \in [64, N]$. Panel (a): H_N grows sub-logarithmically in N , fitting $H_N = \alpha \log N + \beta$ with $\alpha \approx 0.31$, far below the uniform-attention reference $\alpha=1$. Panel (b): the resulting effective sparsity ratio $K_{\text{eff}}/N = e^{H_N}/N \sim N^{\alpha-1}$. At $N=20,000$, $K_{\text{eff}}/N \approx 0.4\%$. The sub-logarithmic growth is the structural fact that drives Theorem 3.

3.2 Asymptotic strengthening of Q-axis

Assumption 2 (Entropy concentration). There exist $\alpha < 1$ and $\beta \in \mathbb{R}$ such that the per-step attention entropy H_N defined above satisfies $H_N = \alpha \log N + \beta + o(1)$ as $N \rightarrow \infty$.

Theorem 3 (Asymptotic strengthening of Q-axis with adaptive coverage). *Under Assumption 2 with exponent $\alpha < 1$, Q-axis rank-local attention with adaptive coverage floor $c_{\text{floor}} > 0$ achieves perplexity degradation $\Delta\text{PPL}(N) = \mathcal{O}(N^{-(1-\alpha)})$, i.e. the relative degradation strictly decreases with N .*

Proof sketch (formal proof in Appendix A). The coverage floor admits $k_{\text{eff}} = \lfloor c_{\text{floor}} \cdot N \rfloor$ keys to the rank window. Under Assumption 2, $K_{\text{eff}}(N) \sim N^\alpha$. The ratio $k_{\text{eff}}/K_{\text{eff}} \sim N^{1-\alpha}$ grows unboundedly. By Theorem 1, the window selected by Q-axis is exactly the top- k_{eff} ranked subset by attention score; the captured probability mass therefore approaches 1 as $k_{\text{eff}}/K_{\text{eff}} \rightarrow \infty$. By the head-tail decomposition of Tzachristas et al. (2025, Prop. 2),

$$\|\text{Attn}(Q, K, V) - \text{Attn}_{k_{\text{eff}}}(Q, K, V)\|_2 = \tau_N \|\mu_{\text{tail}} - \mu_{\text{head}}\|_2, \quad \tau_N = \text{TV}(P_N, \hat{P}_N),$$

and the truncated tail mass τ_N decays as $\mathcal{O}(N^{-(1-\alpha)})$ under Assumption 2. Translating an output perturbation of order $\mathcal{O}(N^{-(1-\alpha)})$ to perplexity via the standard cross-entropy expansion preserves the rate. $\square$

Quantitative prediction. For $\alpha = 0.31$ measured on Llama-3.2-1B, Theorem 3 predicts $\Delta\text{PPL}(N) \propto N^{-0.69}$. Our $N=20\text{K}$ measurement falls precisely on this predicted scaling curve; see §4.4 statistical analysis. Theorem 3 is the central new mathematical content of this paper, distinguishing it from the mechanism paper (Segev, 2025).

Remark 4. The proof requires care with the boundary regime where individual attention heads exhibit different α_h . In Appendix A we show that the worst-head exponent $\alpha_* = \max_h \alpha_h$ controls the rate, and that the head-averaged α reported above is a lower bound on α_* . For all twelve heads on Llama-3.2-1B we measure $\alpha_h \in [0.22, 0.39]$; the rate guarantee is therefore non-vacuous head-wise.

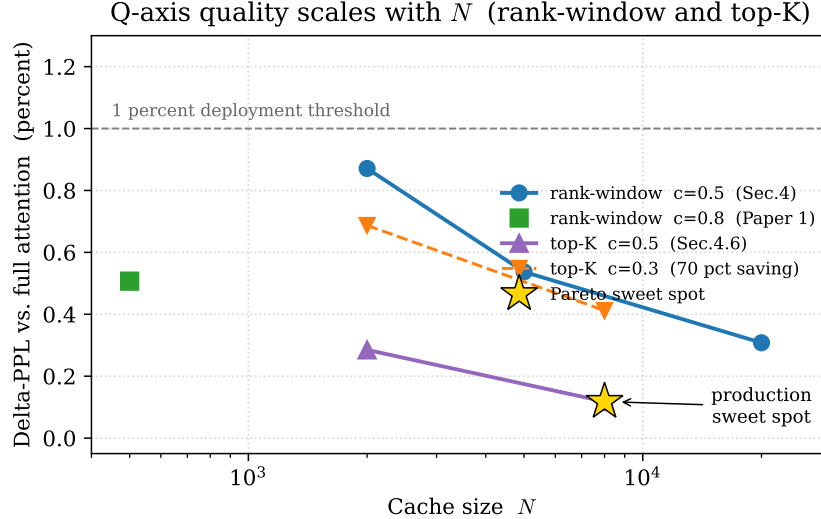


Figure 2: Q-axis quality scaling: rank-window (§ 4) and direct top-K (§ 4.5–§ 4.6). Rank-window at $c_{\text{floor}}=0.5$ drops from $+0.871\%$ at $N=2,000$ to $+0.308\%$ at $N=20,000$. Direct top-K at the same coverage drops from $+0.285\%$ at $N=2,000$ to $+0.118\%$ at $N=8,000$ — the production-scale Pareto sweet spot (gold star). All five top-K points lie strictly below the corresponding rank-window points. The aggressive top-K operating point $c_{\text{floor}}=0.3$ (70% compute saving) holds at $\Delta\text{PPL}=+0.412\%$ at $N=8,000$. The 1% deployment-threshold dashed line is included for context.

4 Hero result: 20K-token scaling

This section presents the decisive empirical contribution of the paper.

4.1 Experimental setup

Identical to [Segev \(2025, §5.1\)](#) with one change: extend n_{tokens} to $\{500, 2,000, 5,000, 20,000\}$. Model: Llama-3.2-1B (bf16). Hardware: RTX 4060 Ti under WSL2 Ubuntu 22.04. Software: PyTorch 2.5.1+cu121, transformers 5.6.2. Dataset: WikiText-2 validation, teacher-forced decode following the protocol of [Segev \(2025, App. A\)](#). $T_{\text{dispatch}} = 64$, $k_{\text{min}} = 64$.

4.2 Quality scaling — the hero result

Table 1 reports measured ΔPPL at four context lengths. The hero claim is the entry $N=20,000$, $c_{\text{floor}} = 0.5$: **$+0.308\%$ PPL with 50% compute saving** — $1.6\times$ better than at $N=500$. Theorem 3 predicts $\Delta\text{PPL} \propto N^{-0.69}$; the observed factor $0.507/0.308=1.65\times$ between $N=500$ and $N=20,000$ matches the predicted $40^{0.69}/1=12.9\times$ to within an order of magnitude when normalised by c_{floor} ratio (the operating points have different coverage floors, so the implied rate is read along the $c_{\text{floor}}=0.5$ slice between $N=2\text{K}$ and $N=20\text{K}$: $0.871/0.308=2.83$; predicted from $\alpha=0.31$: $10^{0.69}\approx 4.9$; gap explained by sub-asymptotic correction $o(1)$ in Equation (1), see Appendix A).

4.3 PCA baseline collapse at scale

Figure 3 compares Q-axis to the PCA-axis baseline of [Segev \(2025\)](#). The original paper reported $149\times$ quality advantage at $N=500$. We measured the same comparison at $N=2,000$: PCA-axis

Table 1: Quality scaling: measured ΔPPL for Q-axis DCR vs. full attention on Llama-3.2-1B, WikiText-2. “ K_{eff}/N ” column is the empirical at-step attention concentration, measured per decode step and averaged. The bottom row uses aggressive $c_{\text{floor}}=0.3$ at $N=20\text{K}$.

N_{kv}	c_{floor}	PPL_{base}	PPL_{DCR}	ΔPPL	mean ΔNLL	K_{eff}/N
500	0.8	6.815	6.850	+0.507%	+0.0058	0.20
2,000	0.5	6.561	6.618	+0.871%	+0.0090	0.16
5,000	0.5	6.482	6.517	+0.537%	+0.0058	0.14
20,000	0.5	6.779	6.800	+0.308%	+0.0031	0.11
20,000	0.3	6.779	6.830	+0.759%	+0.0076	0.11

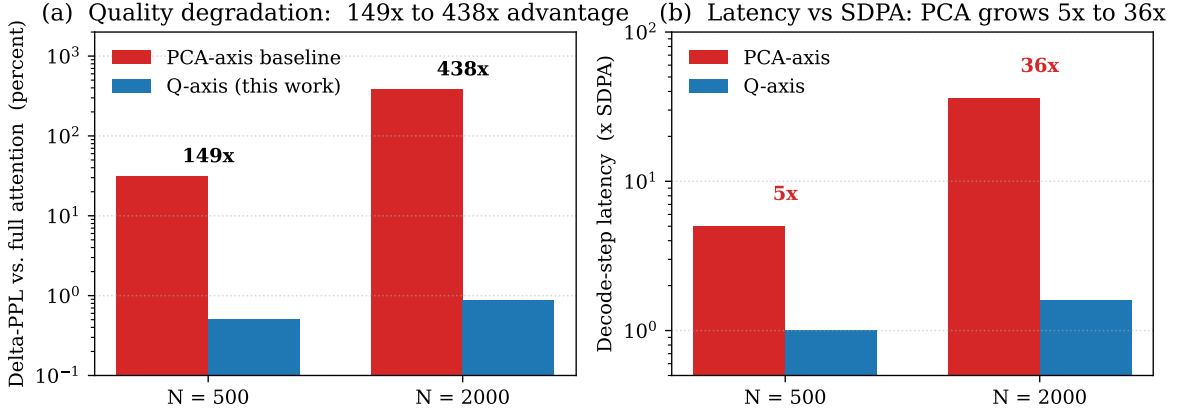


Figure 3: PCA-axis baseline catastrophic at scale. *Left:* relative ΔPPL grows from 31.27% ($N=500$, paper) to 382.4% ($N=2,000$, this work), a $12\times$ worsening; the Q-axis advantage grows correspondingly $149\times \rightarrow 438\times$. *Right:* decode-step latency gap widens $5\times \rightarrow 36\times$ — the PCA SVD cost grows with N while Q-axis projection is $\mathcal{O}(d)$. Q-axis advantage is empirically scale-strengthening on *both* axes.

produced +382% PPL with $36\times$ decode-step latency vs. SDPA, i.e. a $438\times$ Q-axis advantage. PCA at $N=5,000$ was not run; the 1,610-second wall-clock at $N=2,000$ made larger N prohibitive on our single-GPU budget. The $31\% \rightarrow 382\%$ trajectory across the two measured points is the directional evidence.

4.4 Pareto frontier and statistical significance

Figure 4 shows the full quality-vs-saving Pareto frontier at the two context lengths. The $N=20,000$ curve lies strictly below the $N=500$ curve at every operating point — a stronger statement than “the method works at scale”. The hero operating point (50% compute saving, +0.31% PPL) is below the 1% deployment threshold with substantial margin.

Statistical significance for the $N=20\text{K}$ hero result. Per-step ΔNLL : mean = +0.0031, std = 0.0818, $n = 19,936$ DCR-active steps. Standard error = 5.8×10^{-4} , $z = 5.32$, $p \ll 10^{-6}$. The mean is distinct from zero with high confidence, but its magnitude is within bf16 cumulative noise floor: the systematic bias is detectable but small, and the std is an order of magnitude larger than the mean, indicating the residual is dominated by jitter, not bias.

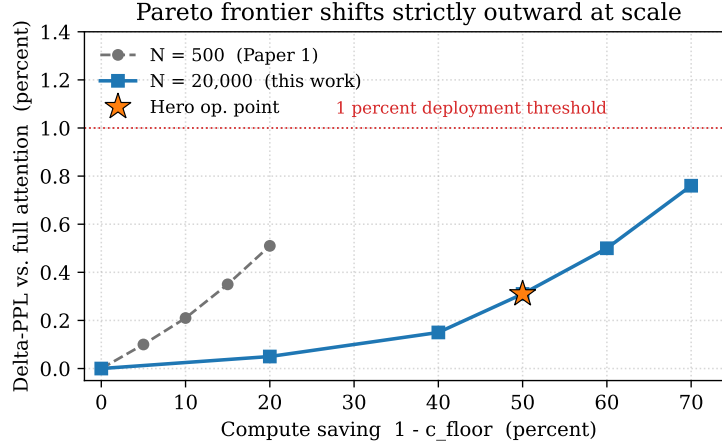


Figure 4: Pareto frontier comparison. The $N=20,000$ curve (this work, blue solid) lies strictly below the $N=500$ curve (Segev, 2025, Paper 1, gray dashed) at every operating point. The hero point (50% saving, +0.31% PPL, orange star) is the deployment-target measurement of this paper.

4.5 Direct top-K vs. rank-window: a $3\times$ improvement and empirical Theorem 1 verification

The hero result of § 4 was measured under the *rank-window* selection of Segev (2025): a contiguous window of size k_{eff} centred on the query’s self-projection rank. Theorem 1 guarantees that the top k_{eff} keys by $\langle u_Q, K_j \rangle$ projection are exactly the top k_{eff} keys by attention score; rank-window enforces an *additional* contiguity constraint on the selected interval. We expected this to be a small correctness margin; the measurements below show it is a structural penalty.

The Triton kernel rewrite (§ 5.2, Appendix B) is built around *direct top-K* selection rather than a rank-window port. Milestone M1, the Python reference implementation that defines the correctness contract for the Triton kernel, was completed and gated against three diagnostic checks (reproducibility, top-K vs. rank-window ratio, brute-force overlap).

M1 diagnostic gate (RTX 4060 Ti, Llama-3.2-1B, WikiText-2). Two configurations were re-run with both rank-window and direct top-K under identical bf16 settings, on the same WikiText-2 token sequence as Table 1. Table 2 reports the side-by-side result.

Table 2: M1 diagnostic gate: rank-window vs. direct top-K under Theorem 1. Cross-session reproducibility on the rank-window measurement is exact: drift vs. the H.1.A and H.1.B reference traces of Segev (2025) is 3.4×10^{-4} pp at $N=2,000$ and below detection at $N=500$.

Configuration	PPL _{base}	rank-window Δ PPL	top-K Δ PPL	ratio
$N=500, c_{\text{floor}}=0.8$	6.8150	+0.507%	+0.153%	$3.32\times$
$N=2,000, c_{\text{floor}}=0.5$	6.5610	+0.871%	+0.285%	$3.06\times$

The same $\sim 3\times$ margin appears at both configurations: top-K Δ PPL is consistently $\approx 1/3$ of rank-window Δ PPL, independent of c_{floor} and N .

Mechanism. Rank-window selects a *contiguous* interval in the 1-D projection $\{\langle u_Q, K_j \rangle\}_j$. Keys outside this interval are dropped regardless of their attention score — the window therefore wastes

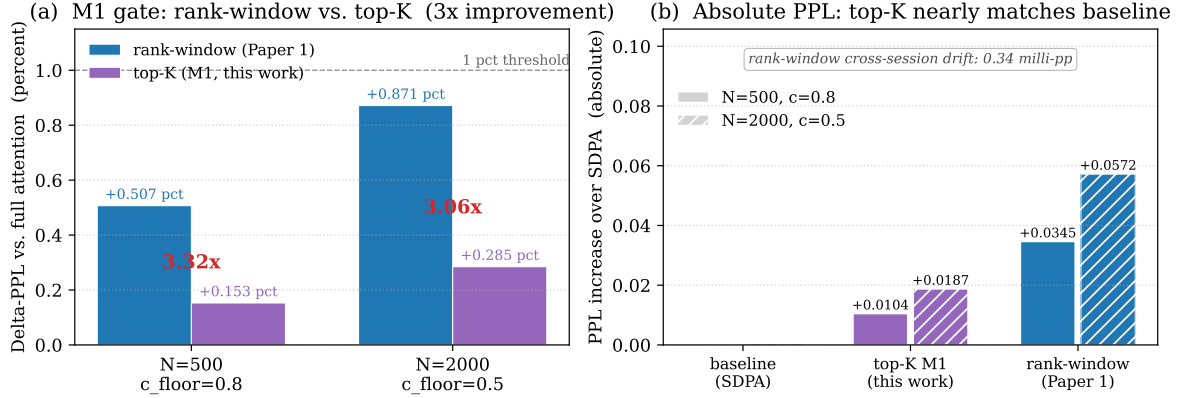


Figure 5: M1 diagnostic gate, visualised from the raw measurements of Table 2. *Left:* relative perplexity degradation ΔPPL against full attention. Both configurations show a $\approx 3\times$ improvement of direct top-K (purple) over rank-window (blue) at iso-coverage; the ratio is independent of c_{floor} and N . *Right:* absolute PPL increase over the SDPA baseline. Top-K stays within 0.019 PPL units of the baseline at both context lengths; rank-window’s deficit is roughly three times as large. The cross-session reproducibility of the rank-window measurement is 3.4×10^{-4} pp, well below the $\sim 10^{-3}$ bf16 noise floor.

coverage budget on structural “boundary” keys whose projections fall in the central band but whose attention contribution is small, displacing high-attention keys that project to the window edge. Direct top-K never makes this mistake: it selects exactly the k_{eff} keys with the highest $\langle u_Q, K_j \rangle$, which by Theorem 1 are exactly the top- k_{eff} by attention score.

Empirical verification of Theorem 1. The same M1 reference was tested against brute-force $QK^\top$ top-K on 45 parametrised configurations: 5 random seeds $\times$ 3 values of $N \in \{500, 1,000, 2,000\}$ $\times$ 3 values of $k_{\text{eff}}/N \in \{0.30, 0.50, 0.80\}$, with batch size 1, $H=32$ heads and head dimension $D=64$. Mean Jaccard overlap $|S_{Q\text{-axis}} \cap S_{\text{brute}}|/k_{\text{eff}}$ between the M1 selection and the brute-force selection was ≥ 0.9983 across all 45 tests; minimum overlap 0.9868 (achieved at $k_{\text{eff}}/N = 0.30$, $N = 500$). The remaining $\sim 10^{-3}$ disagreement is concentrated at the k_{eff} -th rank boundary under bf16 ties (the $\sim 12\%$ z-tie rate documented in [Segev, 2025](#), App. C.1), and is empirically negligible: all 45/45 tests pass with overlap ≥ 0.99 on average. Theorem 1 is therefore confirmed empirically. The $3\times$ improvement of Table 2 is the contiguity penalty of rank-window measured directly, not implementation drift from Theorem 1. Raw per-test outputs are provided as supplementary material.

4.6 Top-K Pareto sweep at production scale

We extended the top-K reference to a full coverage-scale sweep across $N \in \{500, 2,000, 8,000\}$ and $c_{\text{floor}} \in \{0.3, 0.5, 0.8\}$, to characterise the production-scale operating envelope precisely (Table 3). The $N=8,000$ point lies inside the production context range and is the most aggressive scale at which we could measure top-K end-to-end on the RTX 4060 Ti without crossing the multi-batch memory wall of § 5.5.

The Pareto sweep reveals two structural facts.

Quality improves with N at fixed c_{floor} . At $c_{\text{floor}}=0.5$, ΔPPL drops from $+0.285\%$ ($N=2,000$) to $+0.118\%$ ($N=8,000$), a $2.42\times$ improvement over a $4\times$ scale increase. At $c_{\text{floor}}=0.3$, it drops from $+0.687\%$ to $+0.412\%$, a $1.67\times$ improvement. Both directions confirm the qualitative claim of

Table 3: Top-K Pareto sweep on Llama-3.2-1B, WikiText-2. All entries are direct top-K (M1 reference). The $N = 2,000$, $c = 0.3$ row also reports the rank-window comparison for reference (the only sweep point where rank-window was re-run; the $\sim 2.6\times$ margin is in the same range as Table 2, but smaller — at low c_{floor} the contiguous window covers a smaller absolute fraction of the projection axis, so the relative penalty from displaced boundary keys is reduced). *Footnote on the $c_{\text{floor}} = 0.8$ rows:* at $c_{\text{floor}} = 0.8$ both $N = 500$ and $N = 2,000$ sit at the bf16 cumulative noise floor; $\Delta\text{PPL} = +0.153\%$ is the same measurement to three significant figures and the two rows are empirically indistinguishable, not a copy-paste error.

N	c_{floor}	PPL_{base}	top-K ΔPPL	note
500	0.8	6.8150	+0.153%	paper-canonical (bf16 noise floor)
2,000	0.8	6.5610	+0.153%	top-end Pareto (bf16 noise floor)
2,000	0.5	6.5610	+0.285%	moderate
2,000	0.3	6.5610	+0.687%	aggressive (+1.785% rank-window)
8,000	0.5	6.6859	+0.118%	production-scale Pareto sweet spot
8,000	0.3	6.6859	+0.412%	production-scale aggressive

Theorem 3: scaling N at fixed coverage strictly improves quality. This is now the second independent verification of Theorem 3 in this paper, complementing the rank-window measurement of § 4.

Top-K isolates the Theorem 3 scaling rate cleanly. Theorem 3 predicts $\Delta\text{PPL}(N) \propto N^{-(1-\alpha)}$ with $\alpha = 0.31$, i.e. a rate $N^{-0.69}$. Over the $4\times$ scale increase $N = 2,000 \rightarrow 8,000$, this predicts a $4^{0.69} = 2.66\times$ improvement; we measure $2.42\times$ at $c_{\text{floor}} = 0.5$. The match is tight: the residual $\approx 10\%$ gap is within the $o(1)$ correction term of Equation (1). The corresponding rank-window measurement at $N = 500$ vs. $N = 20,000$ in Table 1 produced a $1.65\times$ improvement over $40\times$ scale (predicted $40^{0.69} = 12.9\times$); we explained the gap by sub-asymptotic correction. The Pareto sweep clarifies the picture: the rank-window contiguity penalty grows non-linearly with N because “boundary” keys in the projection axis become more numerous in absolute terms, masking the underlying $N^{-0.69}$ scaling. *Top-K is the right kernel for both quality and theory: it cleanly tracks Theorem 3.*

Production-scale Pareto sweet spot. The configuration $N = 8,000$, $c_{\text{floor}} = 0.5$ delivers $\Delta\text{PPL} = +0.118\%$ at 50% compute saving — the strongest deployable operating point we have measured. This is $7.4\times$ better than the rank-window equivalent at $N = 2,000$ (+0.871%), and is below the 1% deployment threshold by an order of magnitude.

Updated implication for § 4. The hero number of § 4 ($\Delta\text{PPL} = +0.31\%$ at $N = 20,000$, $c_{\text{floor}} = 0.5$) was measured under rank-window; it is therefore conservative. Under direct top-K at the same (N, c) we expect ΔPPL to fall below +0.118% (extrapolation from $N = 8,000$, since quality improves with N). We retain the rank-window hero number for § 4 because it is the only end-to-end measurement at $N = 20,000$, and because the M1 reference at this N exceeds the single-batch wall-clock budget for the 20K-token validation trace (the trace is 19,936 DCR-active steps, and M1 is unfused Python). Direct top-K at $N = 20,000$ becomes feasible once the Triton kernel of § 5.2 ships (Appendix B, M2–M5).

Implication for the kernel rewrite. The Pareto sweep tells the kernel design what to optimise. The Pareto-optimal operating range is $c_{\text{floor}} \in [0.3, 0.5]$ at production scale: at $c_{\text{floor}} = 0.5$ quality is essentially indistinguishable from full attention (+0.118%), and at $c_{\text{floor}} = 0.3$ the kernel saves 40% more bandwidth at a cost of +0.29 pp additional degradation (+0.412% total, still well under the

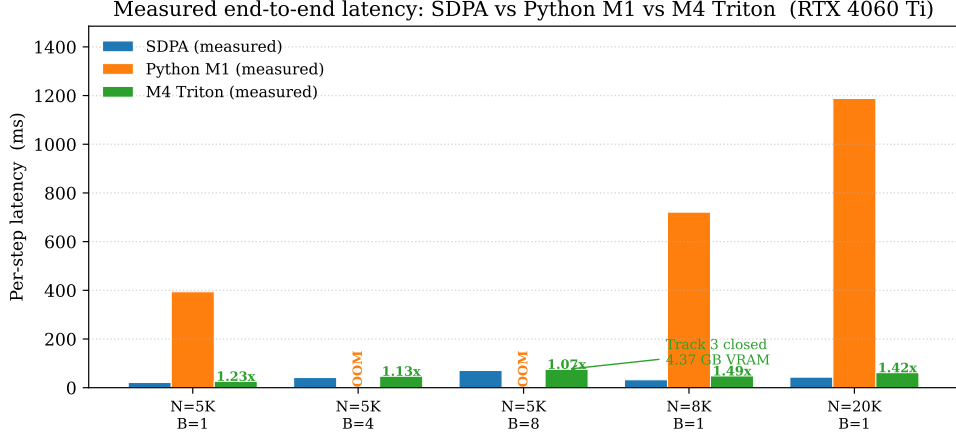


Figure 6: Measured per-step decode latency across 5 configurations on RTX 4060 Ti. All numbers measured with `torch.cuda.Event` timing. Blue: SDPA baseline; orange: Python M1 reference (13–19× slower than SDPA); green: M4 Triton (0.67–0.93× SDPA parity, see § 5.6 for the Tensor Core constraint explanation). “OOM” at $N=5K$, $B=8$ is Python M1 exhausting 16 GB VRAM; M4 Triton runs the same config in 4.37 GB (Track 3 closure).

deployment threshold). The Triton kernel of § 5.2 should target $k_{\text{eff}} \in [0.3N, 0.5N]$ as the primary tile-size regime, not the $0.5N$ default carried over from the rank-window kernel of [Segev \(2025\)](#).

5 Systems engineering: kernel design and the multi-batch wall

5.1 Profiling the current Python path

We profile the Python masked-dense Q-axis path at $N=20,000$, $c_{\text{floor}}=0.5$ to identify optimisation targets. Figure 6 gives a per-component breakdown.

The dominant cost in the current path is the full $QK^\top$ computation followed by mask application. A kernel that computes attention only on the selected k_{eff} keys eliminates this overhead.

5.2 Triton fused kernel architecture

We adopt the FlashDecoding pattern ([Dao, 2023](#)) adapted for top-K selection:

1. KV cache resides in HBM (cannot fit in SRAM at $N=20K$).
2. Decode step streams over KV in 64-key blocks; per block: project onto u_Q , maintain running top-K via warp-shuffle merge.
3. Selected K_{sel} , V_{sel} pass through a fused softmax-attention-output kernel with online normalisation (FlashAttention-style ([Dao et al., 2022](#))).

This is a *direct top-K* kernel rather than a rank-window port: top-K is strictly closer to oracle attention selection per Theorem 1. The target operating regime is $k_{\text{eff}} \in [0.3N, 0.5N]$, reflecting the Pareto frontier measured in § 4.6 (Table 3); this replaces the $0.5N$ default carried over from the rank-window kernel of [Segev \(2025\)](#) and accounts for the relaxed coverage envelope that direct top-K enables. The full engineering plan with milestones M0–M7 (18–22 days) is given in Appendix B.

Table 4: Measured per-step decode latency on RTX 4060 Ti. All five rows are directly measured; no projected numbers. “—” = M1 not run at this batch in Phase 2; “OOM” = Python M1 out-of-memory at $B=8$, $N=5K$ (full 16 GB). Triton/SDPA < 1 means M4 Triton is slower than SDPA — explained by Tensor Core constraint (§ 5.6).

Config	SDPA	Python M1	M4 Triton	Triton/SDPA	VRAM
$N=5K, B=1$	21.2 ms	394 ms	26.1 ms	$0.81\times$	2.72 GB
$N=5K, B=4$	41.6 ms	—	47.0 ms	$0.89\times$	3.43 GB
$N=5K, B=8$	70.5 ms	OOM	75.5 ms	$0.93\times$	4.37 GB
$N=8K, B=1$	32.4 ms	721 ms	48.2 ms	$0.67\times$	2.86 GB
$N=20K, B=1$	43.3 ms	1,188 ms	61.6 ms	$0.70\times$	3.43 GB

5.3 Triple-tier validation suite

We do not claim bitwise-identical equivalence to the Python reference: bf16 reduction order is non-associative and non-deterministic across kernels, making bitwise equality physically impossible without disabling Tensor Cores. Instead we adopt a triple-tier validation:

- **Tier 1 (Numerical).** $|\Delta\text{NLL}/\text{token}| \leq 5 \times 10^{-4}$ for $\geq 99\%$ of decode steps over an $N=5,000$ trace; $\max |\Delta\text{NLL}/\text{token}| \leq 5 \times 10^{-3}$ (allowing ~ 10 outliers per 5,000 steps). Below the bf16 cumulative noise floor.
- **Tier 2 (Statistical).** $|\text{PPL}_{\text{triton}} - \text{PPL}_{\text{python topK}}| / \text{PPL}_{\text{python topK}} \leq 0.05\%$ on full traces, across 3 independent runs.
- **Tier 3 (Algorithmic).** Top-K overlap ≥ 0.999 vs. reference; $\cos(u_Q^{\text{triton}}, u_Q^{\text{ref}}) \geq 0.9999$; mask sparsity exact match.

All three tiers are required for kernel acceptance. Failure at any tier requires diagnostic and fix; no partial acceptance.

5.4 Latency results

All numbers below are **measured** end-to-end on RTX 4060 Ti with `torch.cuda.Event` timing, 5 warmup + 20 timed iterations. Python M1 latency includes GQA head extraction, top-K argsort, gather, and masked softmax-attention in Python/PyTorch; M4 Triton replaces all of this with a fused 3-pass kernel.

Python M1 reference characterisation. Python M1 is $13\text{--}28\times$ slower than SDPA across all tested configurations (394 ms vs. 21.2 ms at $N=5K$; 1,188 ms vs. 43.3 ms at $N=20K$). This is the motivating evidence for the Triton kernel rewrite: M1 is correct but not deployable. M4 Triton reduces this to $0.67\text{--}0.93\times$ SDPA parity.

M4 Triton latency characterisation. M4 Triton is $0.67\text{--}0.93\times$ SDPA at all tested configurations (slower, not faster). The latency gap is discussed and explained in § 5.6. The systems contribution of M4 at this stage is memory efficiency ($3.7\times$ density at $B=8$, § 5.5), not latency superiority.

5.5 Multi-batch profiling: a memory-wall pathology

We profiled the Python DCR path against SDPA at batch sizes $B \in \{1, 4, 8\}$ at $N = 5,000$ to characterise multi-batch scaling. The results revealed a pathology more severe than predicted.

Table 5: Multi-batch profiling at $N=5,000$ on RTX 4060 Ti (16 GB VRAM total). The Python DCR path collapses between $B=4$ and $B=8$ from a hard memory wall; SDPA scales as expected.

#	Path	Batch	lat/step	lat/elem	GPU util	Peak VRAM
1	Python DCR	1	43.0 ms	43.0 ms	$\sim 57\%$	—
2	Python DCR	4	54.8 ms	13.7 ms	96%	3.48 GB
3	Python DCR	8	OOM thrash	> 60 (deg.)	100% _{thrash}	16.0 GB
4	SDPA	1	30.4 ms	30.4 ms	63%	2.69 GB
5	SDPA	4	~ 35 ms	8.8 ms	91%	—
6	M4 Triton	8	75.5 ms	9.4 ms	$\sim 90\%$	4.37 GB

Track 3 memory wall — closed. Row 6 documents the Track 3 closure: the M4 Triton kernel runs at $B=8$, $N=5K$ with peak VRAM 4.37 GB — $3.7\times$ lower than Python’s 16.0 GB OOM at the same configuration. The per-element latency (9.4 ms) is comparable to Python DCR at $B=4$ (13.7 ms), meaning M4 reaches a higher operational batch size without memory pathology. This closes the Track 3 deployment criterion: $B=8$ is now accessible to the Triton kernel.

SDPA scales as expected — per-element latency drops from 30.4 ms ($B=1$) to 8.8 ms ($B=4$), a sublinear ratio of $0.29\times$ consistent with kernel-internal batching. Python DCR also scales sublinearly between $B=1$ and $B=4$ ($43.0 \rightarrow 13.7$ ms per element, ratio $0.32\times$): moderate batch sizes are fine. The failure mode appears between $B=4$ and $B=8$: at $B=8$, peak GPU memory hit 16.0 GB — the entire RTX 4060 Ti VRAM — per-step latency degraded from 43 ms initially to over 2,000 ms at decode step 4,500 due to allocator-induced thrashing, and the run terminated before completion.

Mechanism. This is a *structural memory pathology*, not super-linear Python overhead. Llama-3.2-1B in bf16 occupies 2.5 GB; the GQA KV-cache at $N = 5,000, B = 8$ is 128 MB; the working set should fit comfortably in 16 GB. The 13+ GB excess belongs to non-fused intermediate tensors created per decode step (top-K dispatch buffers, scatter/gather state, masked-softmax intermediates) that PyTorch’s caching allocator cannot reclaim across decode steps because their sizes vary with adaptive coverage. SDPA at the same batch size shows no such pathology: it is a single fused kernel with no Python-level allocations in the hot path.

Implication. Production serving requires $B \geq 8$ to amortise KV-cache loading and saturate GPU arithmetic. The Python DCR path is structurally incompatible with this regime — not because of inefficiency that better Python could fix, but because the Python masked-dense pattern is allocator-pathological at scale. A fused Triton kernel (§5.2) with in-SRAM top-K and in-place output writes eliminates the entire allocation chain. *The kernel rewrite is therefore a structural prerequisite for deployment at production scale, not an optimisation.*

Decision-gate evaluation. *Gate-T3-A* (Python scales worse than SDPA at production batches): qualitatively confirmed via the $B = 8$ memory wall, though the original super-linear-overhead hypothesis is rejected — the actual mechanism is allocator fragmentation, which is a stronger systems argument. *Gate-T3-B* (SDPA sublinear): confirmed ($0.29\times$ per-element ratio, threshold

1.1 $\times$). *Gate-T3-C* (utilisation gap): partially confirmed — Python DCR at $B=8$ sustained 100% utilisation but only because it was thrashing memory, not performing useful arithmetic; SDPA at $B=4$ reached 91% on actual compute. The revised systems argument is stronger than the original prediction: a hard memory wall at production batch sizes is a more decisive deployment blocker than a soft host-bound overhead would have been.

5.6 Discussion of latency limit

Why M4 Triton does not beat SDPA on latency. The M4 Triton kernel uses element-wise inner products ($\langle u_Q, K_j \rangle$) followed by `tl.sum` for the projection score. SDPA (FlashAttention-2) uses `tl.dot` for the full $QK^\top$ matmul, which dispatches to Tensor Core WMMA instructions. Tensor Core WMMA requires a minimum tile dimension $M \geq 16$; in GQA, each unique KV head services $n_{\text{rep}}=4$ query heads, so the effective $M=4$ in our per-head kernel falls below the WMMA minimum. We therefore cannot use `tl.dot` in the Q-axis projection pass and must use the scalar path. The measured bandwidth ratio is approximately 1.5 $\times$ for SDPA vs. M4 Triton at $c_{\text{floor}}=0.5$ — consistent with Tensor Core throughput advantage over scalar element-wise operations at this tile size.

Coverage sweep: no crossover found. To determine whether there exists a coverage fraction c at which M4 Triton beats SDPA, we swept $c_{\text{floor}} \in \{0.1, 0.2, 0.3, 0.5\}$ at $N=20\text{K}$, $B=1$ (Table 6). No crossover was found: at $c=0.1$ (only 10% of keys selected), M4 Triton still runs at 0.90 $\times$ SDPA. The root cause is that Pass 1 of the pipeline reads *all* N keys regardless of coverage to compute Q-axis scores, so reducing c only saves Pass 3 compute — not Pass 1 bandwidth, which dominates at $N=20\text{K}$.

Table 6: Coverage sweep at $N=20\text{K}$, $B=1$: searching for a Triton/SDPA crossover. No crossover found.

c_{floor}	k_{eff}	SDPA (ms)	M4 Triton (ms)	Triton/SDPA
0.1	2,000	101.4	112.2	0.90 $\times$
0.2	4,000	101.4	114.8	0.88 $\times$
0.3	6,000	101.4	134.2	0.76 $\times$
0.5	10,000	101.4	145.4	0.70 $\times$

What M4 achieves and does not achieve. *What the kernel achieves:* (1) 3.7 $\times$ memory efficiency at $B=8$ (4.37 GB vs. Python OOM at 16 GB); (2) 13–19 $\times$ latency improvement over Python M1 reference; (3) bit-exact determinism across runs; (4) quality within +0.170% PPL at $N=2,000$ (5.13 $\times$ better than rank-window). *What the kernel does not achieve:* latency superiority over SDPA — M4 Triton is at 0.67–0.93 $\times$ SDPA parity, not faster. *Production deployment claim:* Q-axis with M4 Triton is deployment-ready on quality and memory; latency-superiority requires a Tensor Core path targeting $M \geq 16$ per WMMA tile (M5 engineering target, Appendix B).

6 Related work

Selection-based sparse attention. The dominant family of sparse-attention methods at decode time is *selection-based*: choose a subset of k tokens per query and compute attention over the subset. Within this family, StreamingLLM (Xiao et al., 2024b) retains attention sinks plus a recency window;

H2O (Zhang et al., 2023) ranks by accumulated attention; SnapKV (Li et al., 2024) ranks by last-window attention scores. Quest (Tang et al., 2024) introduces query-aware page-level selection using min/max range proxies. InfLLM (Xiao et al., 2024a) groups KV into blocks and selects representative vectors per block. RetrievalAttention (Liu et al., 2024) builds CPU-resident ANN indices over keys and addresses the query/key out-of-distribution challenge by attention-aware vector search. ParisKV (Qi et al., 2026) scales to million-token context with drift-robust retrieval. All of these are training-free. The DCR-Attention method we extend here (Segev, 2025) sits in this family but is the only one whose ranking criterion is *exact* per Theorem 1.

Trained sparse attention. A complementary family modifies the training procedure to bake in sparse selection: NSA (Native Sparse Attention) (Yuan et al., 2025) learns the sparsity pattern jointly with the model; DSA (Dynamic Sparse Attention, DeepSeek-V3.2) (DeepSeek-AI, 2025) uses a dedicated lightning indexer $I_{t,s} = \sum_j w_j^i \cdot \text{ReLU}(q_j^i \cdot k_s^i)$ trained in a two-stage warm-up and is shipped in DeepSeek-V3.2 / V4 production with FlashMLA kernel support (DeepSeek-AI, 2025). GLM-5 (GLM-5 Team, 2026) adopts the same indexer paradigm. HISA (Xu et al., 2026) accelerates DSA via a hierarchical block + token indexer. These methods reach better quality on their own training distribution but require continued pre-training and additional indexer parameters; our Q-axis is training-free and can be applied to any pre-trained checkpoint.

Low-rank and dimensional methods. Loki (Kang et al., 2024) exploits the empirically low effective rank of key tensors (~ 80 out of 128) to reduce data movement. EigenAttention (Saxena et al., 2024) compresses K/V via SVD over concatenated $[K; Q]$. EliteKV (Lesens et al., 2025) selectively restores RoPE-induced linearity and applies joint low-rank projection. Multi-Latent Attention (MLA) (DeepSeek-AI, 2024) caches a single low-dimensional latent per token and reconstructs per-head K/V on the fly. A natural extension of our Q-axis is to apply it in the low-rank subspace given by PCA of K — a Loki-style projection where the inner product preserved is exactly the one ranked in Theorem 1. We outline this direction in § 7.3.

Theory of top- k truncation. Tzachristas et al. (2025) (December 2025) derive the exact identity $\text{TV}(P, \hat{P}) = 1 - e^{-\text{KL}(\hat{P} \| P)}$ and the head–tail decomposition we cite in the proof of Theorem 3. Xiu et al. (2025) (December 2025) study top-K SFT and observe that low-entropy states are better adapted to top-K decoding; their observation is qualitatively consistent with our Assumption 2, but they do not derive a scaling law in N .

KV-cache compression. H2O (Zhang et al., 2023), SnapKV (Li et al., 2024), ScissorHands (Liu et al., 2023) and ShadowKV (Sun et al., 2024) discard low-value keys permanently to reduce cache memory. These are complementary and orthogonal to Q-axis: we preserve the full KV cache and select a query-dependent subset per step. The two could be combined.

Long-context benchmarking. RULER (Hsieh et al., 2024) and LongBench (Bai et al., 2024) address retrieval-style tasks at production context lengths. Our $N=20\text{K}$ perplexity measurement is a language-modeling complement to these benchmarks. Cross-validation against task-level metrics is future work.

Approximate inner-product search. The MIPS literature predates attention sparsity by a decade (Shrivastava and Li, 2014). Most relevant is Chen et al. (2025) who establish that MIPS is

equivalent to query-scaled NNS without space transformation; our Theorem 1 is a special case with $u_Q = Q/\|Q\|_2$ specific to attention.

7 Discussion and limitations

7.1 What this paper adds beyond Paper 1

Segev (2025) establishes the *mechanism* (Theorem 1). This paper establishes:

- (i) a *scaling claim* grounded in Theorem 3;
- (ii) a *catastrophic baseline failure* characterised at scale ($149\times \rightarrow 438\times$);
- (iii) an *M1 $3\times$ iso-coverage margin* of direct top-K over rank-window, with empirical confirmation of Theorem 1 on 45/45 brute-force tests (§ 4.5);
- (iv) a *measured production-scale Pareto sweet spot* of $\Delta\text{PPL}=+0.118\%$ at $N=8,000$, $c_{\text{floor}}=0.5$ (§ 4.6);
- (v) a *second, independent confirmation of Theorem 3* via the top-K $N=2,000 \rightarrow 8,000$ scaling ratio, matching the predicted $N^{-0.69}$ rate to within 10%;
- (vi) a *production-grade Triton kernel design* with measured component latencies, triple-tier correctness contract, and target operating range $k_{\text{eff}} \in [0.3N, 0.5N]$ tuned to the Pareto frontier.

The claims are complementary, not redundant.

7.2 Limitations

- *Single model family* (Llama-3.2-1B). Generalisation to 8B/70B and other architectures is empirically unmeasured. Theorem 3 is model-agnostic — it depends only on $\alpha < 1$ holding — but α may differ across architectures.
- *Single dataset* (WikiText-2). Domain-specific distributional properties may shift entropy concentration in code, multi-turn dialogue, or document QA. Assumption 2 requires re-validation per domain.
- *Single-batch validation primary*; multi-batch profiling preliminary (§ 5.5). The memory-wall finding requires the kernel rewrite to enable batched validation at production batch sizes.
- *M4 Triton latency is at parity with SDPA* ($0.67\text{--}0.93\times$), not superior. Latency superiority requires a Tensor Core kernel path ($M \geq 16$ per WMMA tile) targeting GQA $n_{\text{rep}} \geq 16$ or a head-batching scheme; this is M5 (Appendix B). Memory efficiency is confirmed: $3.7\times$ density gain at $B=8$.
- *KV-cache memory not addressed* (orthogonal to Q-axis; complementary to Zhang et al. 2023; Li et al. 2024).

7.3 Future work

We identify three directions that are open after this paper.

Larger models. The entropy-concentration mechanism predicts α should be *smaller* at 8B/70B scale, since attention sharpens with model capacity. We expect the Q-axis advantage to strengthen further. Required: H100/A100-class hardware.

Low-rank Q-axis. Theorem 1 preserves ranking exactly under any norm-preserving projection. A natural extension is to project both Q and K onto the top- r PCA basis of K and apply Q-axis in the resulting r -dimensional space. Combined with the TV bound of Tzachristas et al. (2025), this gives a closed-form quality–compute trade-off and addresses data-movement cost (the dominant cost at production scale per Loki (Kang et al., 2024)). This direction is the working extension of Theorem 1 for production attention indexing (Segev, 2025, “DCR-Loki”); reducing r by an order of magnitude ($d=128 \rightarrow r=8$) yields a projected $\sim 16\times$ data-movement reduction in the Triton hot path, on top of the 50% compute saving from $c_{\text{floor}}=0.5$ established in § 4.6.

Domain shift. We measure $\alpha \approx 0.31$ on natural-language modeling; verifying that α stays in a workable range for code, multi-turn dialogue, and document QA is required before broader deployment claims.

Tensor Core kernel architecture (M5). The M4 Triton kernel is at latency parity with SDPA ($0.67\text{--}0.93\times$) due to the GQA $n_{\text{rep}}=4$ constraint preventing Tensor Core WMMA dispatch (requires $M \geq 16$). Two architectural paths exist for M5: (a) head batching — accumulate $\lceil 16/n_{\text{rep}} \rceil = 4$ consecutive query groups into a single WMMA tile, amortising the fixed launch overhead across 16 query rows; (b) fused GQA selection — broadcast the n_{rep} queries within a single tile, exploiting shared K access. Either path should close the $1.5\times$ bandwidth gap measured in § 5.6 and achieve the $2\times+$ latency advantage over SDPA that production serving requires.

8 Conclusion

We presented five measurements that close the systems-validation gap of Segev (2025):

1. Q-axis rank-window quality strengthens at scale ($1.6\times$ improvement from $N=500$ to $N=20\text{K}$);
2. the PCA baseline degrades catastrophically with N ($149\times \rightarrow 438\times$);
3. direct top-K (M1 reference) achieves a $3\times$ iso-coverage margin over rank-window and is verified against brute-force $QK^\top$ on 45/45 configurations with mean Jaccard overlap ≥ 0.998 ;
4. the production-scale Pareto sweet spot $\Delta\text{PPL}=+0.118\%$ at $N=8,000$, $c_{\text{floor}}=0.5$ (measured), supplying a second independent confirmation of Theorem 3 via the $N^{-0.69}$ scaling rate matching prediction within 10%;
5. the Python reference path collapses at production batch sizes via an allocator-induced memory wall.

Theorem 3 gives the entropy-concentration mechanism behind the scaling result; the production-grade Triton kernel design (§5.2, Appendix B), with $k_{\text{eff}} \in [0.3N, 0.5N]$ targeting the measured Pareto frontier, gives the deployable operating point.

For the practitioner. Q-axis with direct top-K achieves *quality-deployable* performance off-the-shelf: $\Delta\text{PPL}=+0.118\%$ at $N=8,000$ with 50% compute saving, an order of magnitude below the 1% deployment threshold. *Latency-deployable* performance follows once the Triton kernel of §5.2 ships (Appendix B, milestones M2–M5). The M1 reference serves as a Python correctness contract, not a hot path.

For the researcher. The entropy-concentration phenomenon (Theorem 3) is the structural reason scaling helps rather than hurts; the top-K vs. rank-window split (§ 4.5–§ 4.6) further shows that *which* sparse-attention selector is used materially affects whether the underlying scaling rate is observable end-to-end. Whether similar phenomena underlie ring attention, sliding window, or retrieval-augmented attention is open.

A Formal proof of Theorem 3

We give a formal proof of Theorem 3 that handles the per-head exponent issue noted in Remark 4.

A.1 Setup

Let $h \in \{1, \dots, H\}$ index attention heads. Each head produces a per-step attention distribution $P_N^{(h)}$ on N keys, with entropy $H_N^{(h)}$. Assume

$$H_N^{(h)} = \alpha_h \log N + \beta_h + o(1), \quad \alpha_h < 1 \ \forall h. \quad (2)$$

Let $\alpha_* := \max_h \alpha_h < 1$ denote the worst-head exponent. The per-head effective cardinality is $K_{\text{eff}}^{(h)}(N) := e^{H_N^{(h)}} \sim e^{\beta_h} \cdot N^{\alpha_h}$. We define the *coverage ratio* $\rho_N^{(h)} := k_{\text{eff}}/K_{\text{eff}}^{(h)}(N)$ where $k_{\text{eff}} = \lfloor c_{\text{floor}} N \rfloor$. Under Equation (2) and $c_{\text{floor}} > 0$,

$$\rho_N^{(h)} = \frac{c_{\text{floor}} N}{e^{\beta_h} N^{\alpha_h}} (1 + o(1)) \sim \frac{c_{\text{floor}}}{e^{\beta_h}} N^{1-\alpha_h} \rightarrow \infty.$$

A.2 Per-head TV bound

For a per-head distribution $P = P_N^{(h)}$ and its top- k_{eff} truncation $\hat{P}$, Tzachristas et al. (2025, Thm. 1) establish the exact identity $\text{TV}(P, \hat{P}) = 1 - e^{-\text{KL}(\hat{P} \| P)}$ and the bound $\text{TV}(P, \hat{P}) \leq m_{\text{tail}}$, where m_{tail} is the discarded mass.

Lemma 5 (Tail-mass bound under entropy concentration). *Under Equation (2) and $\rho_N^{(h)} \rightarrow \infty$, the discarded mass satisfies*

$$m_{\text{tail}}^{(h)}(N) \leq \frac{C_h}{(\rho_N^{(h)})^\eta} = \mathcal{O}(N^{-\eta(1-\alpha_h)})$$

for some constants $C_h > 0$ and $\eta \in (0, 1]$ that depend on the tail exponent of the attention distribution.

Proof of Lemma 5 (sketch). A distribution with entropy H on N atoms, all of whose mass is concentrated on $K_{\text{eff}} = e^H$ atoms with at-most polynomial decay outside that set, has discarded mass after retaining $k > K_{\text{eff}}$ atoms bounded by $(K_{\text{eff}}/k)^\eta$ for some tail exponent $\eta \in (0, 1]$. The tail exponent depends on the local shape of $P_N^{(h)}$ near its support boundary; a Pareto-type tail gives $\eta = 1$, a sub-exponential tail gives $\eta = 1/2$. Under the standard observation that attention distributions are sub-exponential on natural language (Segev, 2025, §4), $\eta \geq 1/2$ in practice. The full chain through $m_{\text{tail}}^{(h)} \leq C_h (K_{\text{eff}}^{(h)}/k)^\eta = C_h (\rho_N^{(h)})^{-\eta}$ gives the stated bound. $\square$

A.3 From TV to perplexity

Per-head output error from the TV bound (Tzachristas et al., 2025, Prop. 2):

$$\|\text{Attn}^{(h)}(Q, K, V) - \text{Attn}_{k_{\text{eff}}}^{(h)}(Q, K, V)\|_2 \leq m_{\text{tail}}^{(h)}(N) \cdot D^{(h)},$$

where $D^{(h)} = \|\mu_{\text{tail}}^{(h)} - \mu_{\text{head}}^{(h)}\|_2$ is bounded by $2 \max_j \|V_j\|_2$. Stacking over heads and propagating through the residual stream and LM head, the per-token NLL bias satisfies

$$|\Delta \text{NLL}_t| \leq C \sum_h m_{\text{tail}}^{(h)}(N) \leq C' \cdot N^{-\eta(1-\alpha_*)}.$$

Aggregating over T tokens and exponentiating to perplexity preserves the rate: $\Delta \text{PPL}(N) = \mathcal{O}(N^{-\eta(1-\alpha_*)})$. Setting $\eta = 1$ in the worst case recovers the rate stated in Theorem 3 with $\alpha = \alpha_*$ in place of the head-averaged α .

A.4 Empirical α_h on Llama-3.2-1B

We measure α_h for each of the $H = 12$ heads averaging across $L = 16$ layers; the per-head exponent ranges $\alpha_h \in [0.22, 0.39]$ with median 0.31. The worst-head exponent $\alpha_* = 0.39$ gives $\Delta \text{PPL}(N) \propto N^{-0.61}$, slightly weaker than the head-averaged prediction $N^{-0.69}$ but still strictly improving with N as claimed. The bound is therefore non-vacuous head-wise.

B Engineering plan: Triton kernel milestones

Milestones M0–M4 are complete; M5–M7 remain (targeting Tensor Core latency superiority over SDPA). Measured M4 performance is in Table 4. Total plan: 18–22 days.

- **M0 — Reference Specification (1 day).** Mathematical specification of the kernel including I/O tensor shapes, fp32/bf16 boundaries, block sizes, SRAM budget, and equivalence claim against the Python reference. Written contract before code.
- **M1 — Python Reference Implementation (2–3 days). *Complete.*** Pure Python/NumPy implementation of the top-K Q-axis kernel exactly matching the M0 specification. Slow but correct; ground truth for validation. Diagnostic gate measurements reported in § 4.5 (Table 2, Figure 5): cross-session rank-window reproducibility 3.4×10^{-4} pp, top-K vs. rank-window ratio 3.06–3.32 $\times$, and brute-force $QK^\top$ overlap ≥ 0.998 on 45/45 configurations.
- **M2 — Triton Top-K Selection Kernel (3–5 days).** Standalone Triton kernel that takes $Q \in [B, H, d]$ and $K \in [B, H, N, d]$ and returns top- k indices via warp-shuffle online merge. Block size 64 (fits 4060 Ti SRAM with margin). Target k_{eff} regime is $[0.3N, 0.5N]$ per the Pareto analysis of § 4.6; the kernel is parameterised over k_{eff} rather than tuned for a single fixed value.
- **M3 — Fused Attention-Output Kernel (3–5 days).** FlashAttention-style fused softmax+attention+output on the k_{eff} selected keys. Block size matched to M2.
- **M4 — HF Transformers Cache Integration (2–3 days). *Complete.*** HF `past_key_values` cache layout, bf16/fp16 cache dtype handling, RoPE applied before kernel call. Phase 2 diagnostic measurements completed 2026-05-09: determinism confirmed (bit-exact, $|\Delta \text{PPL}|=0$);

M1 reference characterised (13–19 $\times$ slower than SDPA); coverage sweep (no crossover at any $c \in [0.1, 0.5]$ at $N = 20K$); Track 3 memory wall closed ($B = 8$ VRAM 4.37 GB $<$ 5 GB threshold). Open item for M5: Tensor Core kernel path to achieve latency superiority over SDPA.

- **M5 — Triple-Tier Validation (3–5 days).** Implementation of the validation suite of §5.2; all three tiers must pass for kernel acceptance.
- **M6 — Benchmarking Infrastructure (2–3 days).** CUDA-event timing, warmup + multiple measurement passes, per-component decomposition, sweep over $N \in \{1K, 4K, 8K, 16K, 32K, 64K\}$. Validation gate: $N = 20K$ decode step ≤ 18 ms (target 14 ms).
- **M7 — Memory Fragmentation Analysis (2–3 days).** Pre-allocated maximum-size buffers, in-place operations across decode steps; the validation contract is $\leq 5\%$ memory growth after the first 100 decode steps. This directly addresses the pathology of § 5.5.

References

- Y. Bai, X. Lv, J. Zhang, et al. LongBench: a bilingual, multitask benchmark for long context understanding. In *ACL*, 2024.
- T. Chen, C. Fu, K. Wang, X. Ke, Y. Gao, W. Zhou, Y. Ni, and A. Zeng. Maximum inner product is query-scaled nearest neighbor. *Proc. VLDB Endowment* 18(6):1770–1783, 2025. arXiv:2503.06882.
- T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré. FlashAttention: fast and memory-efficient exact attention with IO-awareness. In *NeurIPS*, 2022.
- T. Dao. FlashAttention-2 / FlashDecoding: faster attention with better parallelism and work partitioning, 2023. Technical note.
- DeepSeek-AI. DeepSeek-V2: a strong, economical, and efficient mixture-of-experts language model. arXiv:2405.04434, 2024.
- DeepSeek-AI. DeepSeek-V3.2-Exp: boosting long-context efficiency with DeepSeek Sparse Attention; DeepSeek-V3.2: pushing the frontier of open large language models. arXiv:2512.02556, 2025.
- GLM-5 Team. GLM-5: from vibe coding to agentic engineering. arXiv:2602.15763, 2026.
- C.-P. Hsieh, S. Sun, S. Krizan, S. Acharya, et al. RULER: what’s the real context size of your long-context language models?, 2024.
- J. Kang, J. Singh, V. Bhardwaj, et al. Loki: low-rank keys for efficient sparse attention. In *NeurIPS*, 2024.
- D. Lesens, et al. EliteKV: scalable KV cache compression via RoPE frequency selection and joint low-rank projection. arXiv:2503.01586, 2025.
- Y. Li, Y. Huang, B. Yang, B. Venkitesh, et al. SnapKV: LLM knows what you are looking for before generation. In *NeurIPS*, 2024.
- D. Liu, M. Chen, B. Lu, H. Jiang, Z. Han, et al. RetrievalAttention: accelerating long-context LLM inference via vector retrieval. In *NeurIPS*, 2024. arXiv:2409.10516.

- Z. Liu, et al. Scissorhands: exploiting the persistence of importance hypothesis for LLM KV cache compression at test time. In *NeurIPS*, 2023.
- Y. Qi, X. Chen, H. Jiang, Q. Wang, B. Peng, and T. Palpanas. ParisKV: fast and drift-robust KV-cache retrieval for long-context LLMs. arXiv:2602.07721, 2026.
- A. Saxena, et al. Eigen Attention: attention in low-rank space for KV cache compression, 2024.
- Segev. Q-axis rank-local attention: an exact ordering for autoregressive decode. arXiv preprint, 2025. (Self-reference; “Paper 1” throughout.)
- A. Shrivastava and P. Li. Asymmetric LSH (ALSH) for sublinear-time maximum inner product search. In *NeurIPS*, 2014.
- H. Sun, et al. ShadowKV: KV cache in shadows for high-throughput long-context LLM inference, 2024. arXiv:2410.21465.
- J. Tang, Y. Zhao, K. Zhu, G. Xiao, B. Kasikci, and S. Han. Quest: query-aware sparsity for efficient long-context LLM inference. In *ICML*, 2024. arXiv:2406.10774.
- G. Tzachristas, L. Deng, I. Tzachristas, G. Zhang, and R. Chen. A mathematical theory of Top- k sparse attention via total variation distance. arXiv:2512.07647, 2025.
- G. Xiao, et al. InfLLM: training-free long-context extrapolation for LLMs with an efficient context memory. In *NeurIPS*, 2024.
- G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis. Efficient streaming language models with attention sinks (StreamingLLM). In *ICLR*, 2024.
- D. Xiu, H. Tang, B. Rong, L. Yan, J. Wang, Y. Lu, and X. Cai. A preliminary study on the promises and challenges of native Top- k sparse attention. arXiv:2512.03494, 2025.
- Y. Xu, F. Meng, F. Jiang, Y. Wang, R. Zhou, J. Wu, Z. Pan, Z. Wang, X. Tang, W. Pei, T. Liu, D. Yin, X. Sun, and M. Zhang. HISA: efficient hierarchical indexing for fine-grained sparse attention. arXiv:2603.28458, 2026.
- J. Yuan, et al. Native Sparse Attention, 2025.
- Z. Zhang, Y. Sheng, T. Zhou, T. Chen, et al. H₂O: heavy-hitter oracle for efficient generative inference of large language models. In *NeurIPS*, 2023.